Chapter 12 worked solutions • File-system implementation

OS12-E01 • Indexed allocation

1. A pointer block holds4096/4=1024 entries, each addressing4 KiB, giving4 MiB data, plus twelve direct blocks totaling48 KiB.
2. The first double-indirect block has1024 pointers to1024-entry blocks, giving1,048,576 data pointers.
3. Three 4 KiB indirect blocks occupy12 KiB metadata, excluding the inode pointer itself. They may span different index levels. Preserve the units in each multiplication.

OS12-E02 • Crash consistency

1. An unnamed orphan inode/block can remain. Recovery may reclaim it but cannot thereby make create successful.
2. A journal records the transaction before commit, then recovery completes its inode/directory transition together. An incomplete transaction is not applied under the stated redo/no-steal model.
3. No. A successful read can hit volatile cache without any durable write/commit. Power loss can remove those bytes. Atomic metadata transitions and data durability are distinct.

OS12-E03 • Free-block structures

1. Block8's zero bit means allocated. The logical test examines one bit; an uncached disk still transfers storage blocks, not a one-bit I/O.
2. Popping one free block from a cached head can be O(1) metadata work. Finding a contiguous run or rebuilding the whole free set can traverse the chain and incur more I/O. These are different operations.
3. Concurrent claims need a lock or atomic bit operation. Persistent bitmap, inode, and size consistency additionally needs journaling/COW; a memory lock alone cannot make a power-loss transaction atomic. Bitmap lookup is not O(files).

Chapter 12 worked solutions • File-system implementation (continued)

OS12-E04 • Path walk, open tables, and reads

1. Cached B4 gives home→I3. Read B3 to obtain I3's directory B5; read B5 for u→I5; read B6 for notes.txt→I7. Cold metadata reads are B3, B5, B6. Data reads are separate.
2. Offset0 and8192 B cover logical 0/1, so read B8/B9. P1 becomes8192, P2 stays0. B10 remains for a later read.
3. P1 fd4→O1, whose offset is8192 with access/status flags; P2 fd9→O2 at 0. Both refer to inode 7, size 12288, mapping8–10, and shared page cache. Independent opens create two descriptions; one S1 object cannot simultaneously represent independent offsets.
4. No. P2 changes O2 while P1 keeps O1. Dup/fork sharing one description would share its offset. Path lookup, inode cache, system tables, and per-open state must remain separate.

OS12-E05 • Bitmap space and file extents

1. Free positions are7,11,12,13,14,15. First-fit contiguous allocation selects11–13, because allocated8–10 break a run from 7.
2. New bits are0000000100000011, free={7,14,15}. Inode7 gains11–13 and size 24 KiB.
3. Pointers can identify11–13 while the bitmap still advertises them as free. Another allocation can reuse them, causing duplicate allocation. Journal/COW must coordinate bitmap, pointers, size, and data ordering into a recoverable transition.
4. For extent11,12,13, contiguous mapping reads one data block. Embedded linked allocation follows all three blocks. A cached index needs one data read; a cold index adds one index read for two total. Keep bitmap convention, extent, strings, and inode state consistent.

Chapter 12 worked solutions • File-system implementation (continued)

OS12-E06 • Directory indexes and crash recovery

1. Buckets are0{},1{},2{delta, epsilon, eta},3{gamma},4{beta, zeta},5{},6{alpha},7{}. Successful comparisons sum to11, so mean=11/7≈1.571.
2. Chaining can hold more than eight names in eight buckets. A ninth name does not inherently fill it. Load may motivate rehash; open addressing has the one-entry-per-slot limit. Concurrent migration needs locking, versions, or a complete protocol so readers cannot miss half-moved entries.
3. Before durable commit, no-steal retains old home; ignore the uncommitted log. After durable commit, full after-images permit complete metadata redo before mounting, size 24576 with new pointers/bitmap. Replaying an already completed home remains unchanged. Assigning size←24576 does not produce36 KiB, whereas +=12288 is not idempotent.
4. Committed metadata alone does not guarantee user-data contents. Here ordered journaling completes data11–13 before committing metadata. A metadata-only guarantee without that condition can expose undurable data. Evaluate every crash prefix's final visible state.

OS12-E07 • Indirection and sparse boundaries

1. Offset=5243003 gives q=1280, d=123. Subtract12 direct and1024 single blocks:244=0×1024+244. Follow double[0][244], then byte123. Dividing1280 immediately would forget the first1036 blocks.
2. Single covers[12,1036), or12–1035. Double covers[1036,1049612), or1036–1049611. Two cold index reads plus data give three I/Os. A cached first index gives two; both cached give one.
3. A legal hole returns zeros without allocating data. Large logical size can coexist with small allocated space. Upper metadata may still require reads; an already known zero upper pointer ends the walk without an invented data read. Distinguish addressability, index bounds, displacement, and actual I/O.

Chapter 12 worked solutions • File-system implementation (continued)

OS12-E08 • Open, dup, and EOF states

1. Initially P1:4→O1, P2:9→O2, offsets(0,0). Read8192 gives(8192,0). Dup adds P1:6→O1 without changing offsets. Read4096 through6 gives(12288,0). P2 seek gives(12288,4096). The final read returns 0 and leaves(12288,4096).
2. O1.offset equals size, so no bytes remain. Seek changes O2, not an inode-global cursor. Shared inode identity does not imply a shared description.
3. Closing6 removes one O1 reference; fd4 retains another. Fork-inherited fd4 refers to the same O1, so reads/seeks through either process affect the shared cursor. Independent opens instead create O1/O2. Inode, description, and fd-table sharing are different relationships.

OS12-E09 • Redo invariants and non-idempotent replay

1. Before commit, no-steal leaves home=M0; discarding uncommitted records preserves it. After commit, complete durable M1 after-images overwrite bitmap, pointers, and size into M1 despite partial home updates. After durable checkpoint, home is already M1; uncleared log replay remains M1. Log clearing follows durable checkpoint.
2. Two increments produce12288→24576→36864 and duplicate11–13 in the array. Use size←24576, pointers←[8,9,10,11,12,13], and the complete new bitmap. Repeating these assignments equals one application.
3. An inode can point to11–13 while the bitmap still marks them free. Discarding the log leaves the partial home state and allows another file to allocate the same blocks. Prohibit uncommitted home writes or retain undo before-images under a suitable protocol; the no-steal proof no longer applies.
4. No. Incremental B2 contains only post-Monday changes; restore B0→B1→B2 including deletions/renames. A differential B2 would contain all changes since B0. A snapshot sharing its volume/device can fail with that device and does not replace a backup in an independent failure domain. Finite crash-prefix checks support implementation; the general argument uses commit recognition, no-steal, idempotence, and durable ordering.

Chapter 12 worked solutions • File-system implementation (continued)

Verification code appendix: file_allocation.py

```
"""Compute indexed-allocation coverage and simple allocation fault checks.

Run:
    python file_allocation.py
"""

BLOCK_SIZE = 4096
POINTER_SIZE = 4
DIRECT_POINTERS = 12


def indexed_capacity() -> dict[str, int]:
    pointers_per_block = BLOCK_SIZE // POINTER_SIZE
    single_bytes = pointers_per_block * BLOCK_SIZE
    double_pointer_entries = pointers_per_block**2
    double_bytes = double_pointer_entries * BLOCK_SIZE
    direct_bytes = DIRECT_POINTERS * BLOCK_SIZE
    return {
        "pointers_per_block": pointers_per_block,
        "direct_bytes": direct_bytes,
        "single_indirect_bytes": single_bytes,
        "double_pointer_entries": double_pointer_entries,
        "double_indirect_bytes": double_bytes,
    }
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 12 worked solutions • File-system implementation (continued)

```
def allocate_unique(free: set[int], count: int) -> list[int]:
    selected = sorted(free)[:count]
    if len(selected) != count:
        raise RuntimeError("not enough free blocks")
    free.difference_update(selected)
    return selected


def main() -> None:
    caps = indexed_capacity()
    assert caps["pointers_per_block"] == 1024
    assert caps["single_indirect_bytes"] == 4 * 1024 * 1024
    assert caps["double_pointer_entries"] == 1024**2
    free = set(range(8, 14))
    blocks = allocate_unique(free, 4)
    assert blocks == [8, 9, 10, 11]
    assert set(blocks).isdisjoint(free)
    print(caps)
    print("allocated", blocks, "remaining", sorted(free))


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study